

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 08 · LECTURE NOTES

Model-Based Methods

Learn the dynamics, then plan with them.

MODULE OUTCOMES

- Explain the sample-efficiency argument for model-based learning.
- Explain model exploitation and the standard defences.
- Explain Dyna and model-based policy optimisation.
- Explain latent world models and why they work.
- Explain MuZero's planning and what it learns.

1 The sample-efficiency argument

A LEARNED MODEL TURNS ENVIRONMENT STEPS INTO COMPUTE

Module 01 §2 established that environment steps are the binding constraint. A learned model can be queried millions of times for the one-off cost of the data it was fitted on, so the scarce resource is traded for an abundant one.

So planning inside the model, or training a policy on model rollouts, converts sample cost into compute cost — which is **the single strongest argument in this field** and the reason model-based methods have persisted through two decades of not quite working.

And the numbers now support it: model-based methods reach Atari performance in around 100,000 frames that early model-free methods required tens of millions for — a two-order-of-magnitude improvement on the metric that matters most.

Which is roughly the human figure from Module 01 §2 — suggesting that a substantial part of the sample-efficiency gap was the absence of a model rather than anything more mysterious. **A satisfying and only partly complete explanation:** humans also bring priors about objects and causality that no current model learns from scratch.

Use	How	Example
Generate experience	Train an ordinary model-free learner on trajectories sampled from the model.	Dyna (tabular), MBPO (deep) — the simplest and most robust use.
Plan at decision time	Search forward from the current state, choose an action, discard the search.	Monte Carlo tree search — CSCE 625 Module 05 §3; MuZero (§4).

Use	How	Example
Short-horizon control	Optimise a sequence of actions over a few steps, execute only the first, and re-optimize.	Model-predictive control — standard in industrial control long before this field existed.
Differentiate through it	Back-propagate the return through the learned dynamics into the policy.	Dreamer (§3). Powerful and delicate, because model error propagates through the gradient too.

Planning at decision time is the use that composes directly with CSCE 625. Given a model, you are back in that course's setting — a known MDP, with tree search and value iteration available — which means the model-based agent is a learned front end to a classical planner, and that is the cleanest way to understand MuZero (§4).

2 Model exploitation

THE POLICY FINDS WHERE THE MODEL IS WRONG

A policy optimised against a learned model will find the states where the model is most optimistic, because that is where the model promises the most reward — and the optimiser is doing exactly what it was asked to do.

And the model is least accurate where it has least data, which is exactly where an exploring policy wants to go. The two effects compound: the policy is drawn to under-sampled regions both by curiosity and by the model's erroneous optimism there.

Errors also compound over the length of a rollout. A 1% per-step prediction error leaves the predicted state unrecognisable after fifty steps, so long rollouts in a learned model are fiction regardless of how good the one-step accuracy looks.

This is CSCE 669 Module 12 §4's warning at its sharpest. The optimiser seeks the corner of your model least like reality and builds its policy there — and here the model is itself learned and imperfect by construction, so there is always such a corner. Which makes the defences below structural rather than optional.

- **Short rollouts.** Branch five to ten model steps from real states drawn from the replay buffer, rather than rolling out from an initial state — which bounds the compounding directly and keeps every imagined state close to real data. This is MBPO's main idea and it is the highest-value defence.
- **Model ensembles.** Train several models with different initialisations; where their predictions disagree, the model is uncertain — so disagreement is a usable, cheap uncertainty estimate, which is the thing CSCE 753 Module 08 §4 said learned systems do not provide by default.
- **Pessimism.** Penalise the predicted reward by the model's uncertainty, so the policy actively avoids regions the model does not know — the inverse of Module 02 §2's optimism, and correctly so: you are optimistic about unexplored *reality* and pessimistic about unmodelled *predictions*.
- **And re-fit the model continuously.** The policy's exploitation generates exactly the data that corrects the error it exploited — the agent goes to the place the model was wrong about, discovers the truth, and

the model is updated. **So the failure is self-correcting provided you alternate between optimising and collecting**, which is **the elegant part and the reason model-based methods are necessarily iterative** rather than a train-then-deploy pipeline.

3 World models and latent dynamics

PREDICTING PIXELS IS THE WRONG OBJECTIVE. Most of an image's bits are irrelevant to control -- textures, distant scenery, lighting, shadows -- and a pixel-reconstruction loss spends its capacity on exactly those, because they are where most of the bits are.

SO: ENCODE to a compact latent z and learn the dynamics THERE.

```
encoder      z_t = e(o_t)
dynamics     z_{t+1} = f(z_t, a_t)      (recurrent)
reward head  r_t = r(z_t)
and optionally a decoder, used only as a training
              signal to keep z informative about the observation
```

THEN TRAIN THE POLICY ENTIRELY INSIDE THE LATENT MODEL -- thousands of imagined trajectories per real environment step, at no environment cost.

DREAMER goes further: the latent dynamics are DIFFERENTIABLE, so the policy gradient can be back-propagated THROUGH the imagined trajectory rather than estimated by sampling (Module 05).

- far lower gradient variance
- and model error now compounds through the GRADIENT as well as through the state, which is why the imagination horizon is kept short (section 2).

THIS IS WHY MODEL-BASED METHODS FINALLY WORKED: the model does not have to be accurate in pixels, only in the features that predict reward.

The latent-space insight is the one to take from this module. Accuracy is needed only where it affects the decision — which is a general modelling principle well beyond reinforcement learning, and it is the same idea as CSCE 636's representation learning: **a representation is good if it preserves what the task needs and discards what it does not**, and the reconstruction objective is a proxy for that rather than the thing itself.

4 MuZero, and the status of the field

MUZERO LEARNS A MODEL GOOD ONLY FOR PLANNING

It learns three functions: a representation function mapping observations to a latent state, a latent dynamics function, and a prediction head producing a policy, a value, and a reward from the latent state.

And it never reconstructs the observation at all. The latent state has no requirement whatsoever to represent the world — only to make the predictions correct under the dynamics, which is a far weaker and far more achievable requirement.

So the model is trained end to end to support Monte Carlo tree search (CSCE 625 Module 05 §3), and **its internal state may bear no resemblance to anything a person would recognise as a board position or a game state.** That is permitted, and the planning still works.

Which is the point, and it is easy to miss: a model only has to be accurate in the quantities planning actually consumes — reward, value, and policy. **It is §3's principle taken to its conclusion**, and it dissolves the assumption that a model-based agent needs a model *of the world* rather than a model *of the decision problem*.

WHERE MODEL-BASED METHODS STAND

Clearly best on sample efficiency, by one to two orders of magnitude — which is the metric that matters wherever interaction is slow, expensive, or risky, and that is most non-game applications.

More complex to implement and tune: several networks, an imagination horizon, a model-to-real data ratio, and the defences of §2 — all of which are hyperparameters that model-free methods do not have.

And strongest where the dynamics are learnable: board and video games, simulated robotics, and systems with smooth physics. Weaker where the dynamics are chaotic, discontinuous, or involve other adapting agents (Module 11), since a model of a learning opponent is a model of a moving target.

So the honest recommendation is: model-free (PPO or SAC) when samples are cheap, because it is simpler and the complexity buys nothing; model-based when samples are not cheap, because nothing else closes the gap. And if you already have a model, you are in CSCE 625 and should plan rather than learn (Module 01 §3's first row, which keeps being the right answer).

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sutton & Barto — chapter 8 (free PDF)	Dyna and the planning-learning relationship , in the tabular setting where it is clearest.
Janner et al. — When to Trust Your Model: Model-Based Policy Optimization (free)	§2's short-rollout defence , with the analysis that says how short.

Resource	What it covers for this module
Hafner et al. — Dream to Control, and DreamerV3 (free)	§3's latent world model , and DreamerV3 is notable for working across many domains with fixed hyperparameters — which is rare in this field.
Schrittwieser et al. — Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero) (free)	§4's method . Read it for the observation that the model need not model the world.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement Dyna-Q** on a grid world and compare sample efficiency against plain Q-learning.
- 2 **Vary the number of planning steps per real step** and plot performance against environment steps.
- 3 **Learn a dynamics model** for CartPole and measure one-step and fifty-step prediction error.
- 4 **Confirm the compounding** and report the horizon at which the prediction becomes useless.
- 5 **Optimise a policy against the learned model only** and evaluate it on the real environment. Report the gap.
- 6 **Identify where the policy exploited the model** by comparing predicted and actual rewards along its trajectory.
- 7 **Add short rollouts from real states** and report the improvement.
- 8 **Train a model ensemble** and plot disagreement against prediction error. Confirm they correlate.
- 9 **Add an uncertainty penalty** and report the effect on the model-to-real gap.
- 10 **Alternate optimisation and collection** and show the exploited error being corrected.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the sample-efficiency argument for model-based learning.
- 2 Give the Atari figures for model-based, model-free, and human.
- 3 Name four uses of a learned model and which composes with CSCE 625.
- 4 Explain model exploitation and why the two effects compound.
- 5 Why are long rollouts in a learned model fiction?
- 6 Name four defences and say which is self-correcting.
- 7 Why is predicting pixels the wrong objective?

- 8 What does Dreamer add, and what does it cost?
- 9 What does MuZero's latent state have to represent?
- 10 Give the recommendation for when to use model-based methods.